به نام خدا

گزارش پروژه اصلی

اعضای گروه :

عماد معمار – 402522085

سپهر نریمانی – 402522139

پارسا عربشاهی – 402521405

استاد : دکتر سمیه کاشی

زمستان 404

مقدمات :

هدف اصلی پروژه، طراحی و پیاده سازی یک کنترل گر چراغ راهنمایی رانندگی برای یک تقاطع T شکل است.

در این پروژه، ما کنترل رفتار چراغ راهنما را با استفاده از یک FSM انجام میدهیم.

این ماشین حالت به صورت ترتیبی بین این حالات جابجا میشود تا توالی های درست رنگ چراغ هارا تولید کند.

فایل اصلی :

از جزییات طراحی کد به تفصیل در کنفرانس کلاسی گفته شده. بنابراین به یک توضیح خلاصه برای مرور اکتفا میکنیم:
سیستم ما یک سیستم کنترل چراغ راهنمایی هوشمند برای تقاطع بین جاده اصلی و جاده فرعی است. در حالت عادی، چراغ جاده اصلی سبز و چراغ جاده فرعی قرمز است تا ترافیک جاده اصلی به صورت روان عبور کند. وقتی سنسور نصب شده در جاده فرعی وجود خودرو را تشخیص دهد، سیستم وارد عمل می‌شود: ابتدا چراغ جاده اصلی به مدت ۳ ثانیه زرد می‌شود، سپس چراغ اصلی قرمز شده و چراغ جاده فرعی به مدت ۱۰ ثانیه سبز می‌شود تا خودروهای فرعی عبور کنند. پس از آن، چراغ فرعی به مدت ۳ ثانیه زرد شده و در نهایت سیستم به حالت اولیه بازمی‌گردد. این سیستم با استفاده از یک ماشین حالت چهارحالته پیاده‌سازی شده و شامل تایمرهای دقیق برای کنترل زمان‌بندی چراغ‌هاست. برای کار در محیط واقعی، مقادیر تایمر باید با فرکانس کلاک FPGA تنظیم شوند.

فایل تست بنچ :

```
COMPONENT traffic_light_controller
   PORT(
        sensor : IN  std_logic;
        clk : IN  std_logic;
        rst_n : IN  std_logic;
        light_highway : OUT  std_logic_vector(2 downto 0);
        light_farm : OUT  std_logic_vector(2 downto 0)
       );
   END COMPONENT;
```

این دقیقا مشابه همان پورت های کنترلر اصلی ماست.

```
  signal sensor : std_logic := '0';
  signal clk : std_logic := '0';
  signal rst_n : std_logic := '0';
 --Outputs
  signal light_highway : std_logic_vector(2 downto 0);
  signal light_farm : std_logic_vector(2 downto 0);
  constant clk_period : time := 10 ns;
```

اینجا سیگنال های تست بنچ ما تعریف میشوند. سیگنال های ورودی با مقدار اولیه 0 مقداردهی میشوند و سیگنال های خروجی نیازی به این کار ندارند.

دوره تناوب را هم برابر با 10 نانوثانیه یا همان فرکانس 100 مگاهرتز قرار میدهیم.

```
-- Instantiate the traffic light controller
  trafficlightcontroller : traffic_light_controller PORT MAP (
         sensor => sensor,
         clk => clk,
         rst_n => rst_n,
         light_highway => light_highway,
         light_farm => light_farm
```

```
        );
   -- Clock process definitions
```

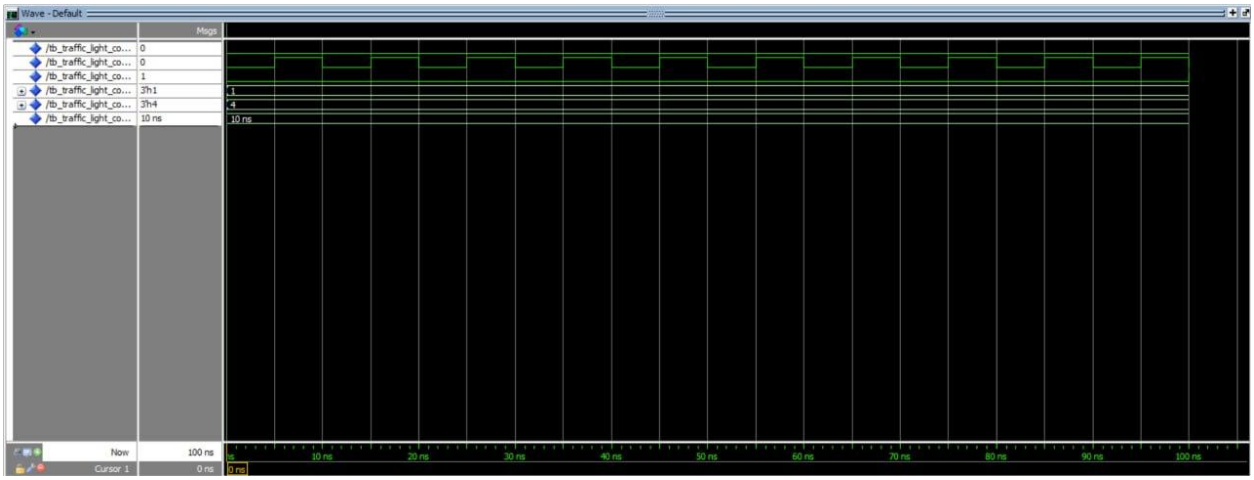
در این قسمت هم سیگنال های تست بنچ را به پورت های واحد تحت تست (uut) متصل میکنیم.

```
stim_proc: process
   begin
    rst_n <= '0';
    sensor <= '0';
    wait for clk_period*10;
    rst_n <= '1';
    wait for clk_period*20;
    sensor <= '1';
    wait for clk_period*100;
    sensor <= '0';
        wait;
   end process;
```

این هم بخش اصلی تست بنچ.

برای simulate کردن پروژه، با استفاده از گزینه simualte در بالای نرم افزار ModelSimاین کار را انجام دادیم. ابتدا تمام فایل ها را کامپایل میکنیم سپس . سپس در بخش simualtion گزینه start simulation را انتخاب کرده و از فولدرwork ، تست بنچ مورد نظر را انتخاب میکنیم، همچنین در قسمت optimization options گزینه apply full visibility to all modulesرا نیز انتخاب میکنیم.

سپس از تب objects تمام متغیرها را انتخاب کرده و به صفحه wave اضافه میکنیم.
اکنون با زدن گزینه Run 100 ns در بخش simulation میتوانیم خروجی را مشاهده کنیم.



این هم waveform نهایی

فرآیند سنتز و فایل بیت استریم:

با قرار دادن فایل eightbit_fulladder به عنوان top mudule ، فایل fulladder به عنوان فایل های زیرین آن و کد ucf که به صورت زیر میباشد(مدل spartan 6 فرض شده)، پروژه به درستی سنتز شده و فایل .bit آن نیز قرار داده شده.

```
NET "A[0]" LOC = P1 | IOSTANDARD = LVCMOS33;
NET "A[1]" LOC = P2 | IOSTANDARD = LVCMOS33;
NET "A[2]" LOC = P5 | IOSTANDARD = LVCMOS33;
NET "A[3]" LOC = P6 | IOSTANDARD = LVCMOS33;
NET "A[4]" LOC = P7 | IOSTANDARD = LVCMOS33;
NET "A[5]" LOC = P8 | IOSTANDARD = LVCMOS33;
NET "A[6]" LOC = P9 | IOSTANDARD = LVCMOS33;
NET "A[7]" LOC = P10 | IOSTANDARD = LVCMOS33;

NET "B[0]" LOC = P11 | IOSTANDARD = LVCMOS33;
NET "B[1]" LOC = P12 | IOSTANDARD = LVCMOS33;
NET "B[2]" LOC = P14 | IOSTANDARD = LVCMOS33;
NET "B[3]" LOC = P15 | IOSTANDARD = LVCMOS33;
NET "B[4]" LOC = P16 | IOSTANDARD = LVCMOS33;
NET "B[5]" LOC = P17 | IOSTANDARD = LVCMOS33;
NET "B[6]" LOC = P21 | IOSTANDARD = LVCMOS33;
NET "B[7]" LOC = P22 | IOSTANDARD = LVCMOS33;

NET "Cin" LOC = P23 | IOSTANDARD = LVCMOS33;

NET "S[0]" LOC = P24 | IOSTANDARD = LVCMOS33;
NET "S[1]" LOC = P26 | IOSTANDARD = LVCMOS33;
NET "S[0]" LOC = P27 | IOSTANDARD = LVCMOS33;
NET "S[1]" LOC = P29 | IOSTANDARD = LVCMOS33;
NET "S[0]" LOC = P30 | IOSTANDARD = LVCMOS33;
NET "S[1]" LOC = P32 | IOSTANDARD = LVCMOS33;
NET "S[0]" LOC = P33 | IOSTANDARD = LVCMOS33;
NET "S[1]" LOC = P34 | IOSTANDARD = LVCMOS33;

NET "Cout" LOC = P35 | IOSTANDARD = LVCMOS33;
```